

Embeder End-to-End Workflow Specification

This document details the step-by-step user and autonomous system workflow in **Embeder**, taking an idea from raw concept through hardware simulation down to physical PCB generation.

1. The Four-Stage Engineering Pipeline

Stage 1: Concept & System Architecture

- User inputs a high-level system goal (e.g., *"I want to build a low-power wireless weather station using an ESP32 and BME280 sensor"*).
- Embeder generates a complete System Architecture Spec, including power requirements, component selection (BOM), microcontrollers, and pin mapping options.
- User reviews and refines parameters in the interactive decision canvas.

Stage 2: Firmware Design & Automated Testing

- Embeder drafts modular hardware-abstracted C/C++ firmware code tailored to the selected target (Arduino, ESP32, RP2040, or STM32).
- The Firmware MCP server compiles the code in a background process using native toolchains.
- If compilation fails, compiler stderr outputs are fed back into the context window for self-healing error resolution.

Stage 3: Hardware Emulation & Logic Simulation

- Embeder launches a local headless simulation instance using Renode / QEMU.
- Peripherals are virtually attached, and firmware execution is validated for timing issues, stack overflows, or pin-state errors.
- Real-time logic analyzer charts and serial terminal logs are rendered in the desktop application interface.

Stage 4: Idea-to-PCB Generation & Fabrication Export

- Embeder constructs schematic connections and passes component placement rules to KiCad via Python automation scripts (``pcbnew``).
- Design Rule Checks (DRC) are executed programmatically to ensure trace clearance, power plane integrity, and copper pour continuity.
- The project exports production-ready files: Gerber files, NC Drill files, Bill of Materials (BOM CSV), and interactive HTML pick-and-place files for PCB assembly.